

Memory Management with ARC

Approaches to Memory Management

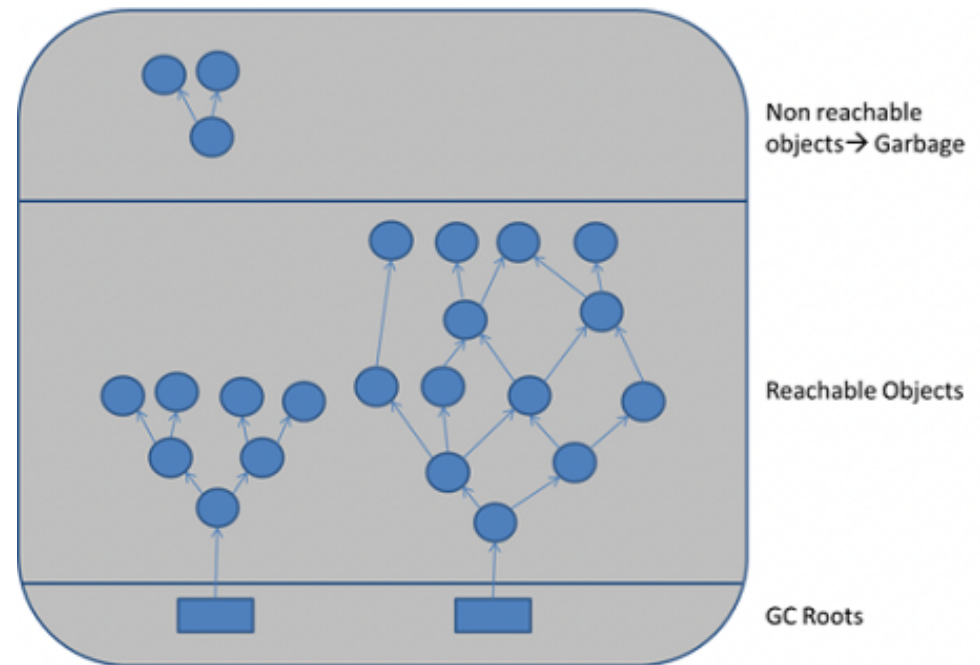
Manual:

- Programmer is responsible for properly allocating and deallocating memory in the heap / free store
- Example: `malloc()` and `free()` in C or `new()` and `delete()` in C++

Approaches to Memory Management

Automated with Garbage Collection

- A collector runs intermittently to find objects that can no longer be reached in the memory graph
- The algorithm traverses all object references and marks every object found as alive.
- Any unreachable objects are marked as garbage.
- Example: Java, C#, Python, most modern languages



Garbage Collection

Disadvantages

Benefits are pretty clear (prevents lots of memory leaks and errors; easier to use), but there are some disadvantages:

- **Overhead:** Cost in memory and processing time to track the memory graph and run the algorithm
- **Unspecified Timing:** Collection algorithm does not run at a time the programmer can plan for (non-deterministic)
- **Stalling/Interruption:** Given the overhead and timing issues, in complex programs or during intricate animations/interactions, garbage collection might stall or interrupt your program

A lot of modern GC implementations have become pretty sophisticated, so these performance hits/stalls have gotten a lot better

Automated Reference Counting

- **Less Overhead:** While there is still some overhead, it is substantially less than using Garbage Collection
- **Retain Count:** ARC tracks how many **strong** references there are to an object. When that count reaches zero, it deallocates the object.
- **Strong vs. Weak vs. Unowned References:** A **strong** reference is just a reference that increase an object's retain count. By default, references are strong unless we tell the compiler otherwise. **Weak** and **unowned** references do not increment the retain count. (We'll talk about unowned later)
- **Deterministic Timing:** There is no separate process running periodically that tries to detect unreferenced objects.
-

Automated Reference Counting

- Essentially, as long as *anything* has a strong reference to an object, it will not be deallocated.
- **Retain cycles** (a kind of memory leak) occur when two objects both have strong references to each other
- **Note:** Since value types are copied, ARC really only applies to *reference types*.

A Note on Weak References

We've already seen some examples of using **weak** references to prevent retain cycles

- **IBOutlets**: They're **weak** because they're basically utilities to make our lives easier. The **strong** reference to any view we put on a Storyboard is from the view hierarchy originating at the View Controller's **root view**.
- **Delegates**: They're **weak** because they're actually a reference to another object, and we don't want to accidentally cause a retain cycle when we sign up to be a TableView's delegate for example.